

Проект по курсу
MS SQL Server Developer

Тема: Проектирование БД для управления личными финансами

Оглавление

1. Общее описание кейса.....	3
2. Цели и назначение.....	3
2.1 Цели проекта.....	3
2.1. Объём MVP.....	3
3. Характеристики системы.....	4
3.1 Словарь.....	4

1. Общее описание кейса

Разрабатываемое решение на базе СУБД MS SQL Server является предназначено для управления личными финансами физического лица (ФЛ) — резидента РФ.

Модель данных опирается на базовые принципы бухгалтерского учёта, принятые в РФ, с отдельными допущениями.

2. Цели и назначение

2.1 Цели проекта

Проектирование базы данных выполняется в рамках разработки проекта по созданию системы управления личными финансами.

Проект ориентирован на создание комплексной системы управления личными финансами.

Цель проекта — разработка БД для анализа и управления личными финансами.

Для понимания роли разрабатываемой базы данных приведём полный скоуп вышестоящего проекта:

- Базы данных по направлениям, включая, но не ограничиваясь:
 - учёт и управление операционной деятельностью (повседневные расходы)
 - учёт и управление инвестиционной деятельностью (долговременные вложения)
 - учёт и управление имуществом (предполагается управление налогооблагаемой имущественной базой)
- Систему классификации данных
- Систему для анализа и принятия решений.

В рамках проекта по курсу MS SQL будет реализовываться MVP (минимальный работоспособный проект) базы данных с аналитическим инструментом для целей моделирования и оценки концептуальной правильности направления разработки.

2.1. Объём MVP

В рамках MVP предполагается реализация:

- Хранение данных по счетам (картам с привязанными счетами):
 - баланс на дату
 - приходно/расходные операции
- Детализация данных по:
 - клиентам (вторая сторона приходно/расходной операции)

- сделкам (аналитическая сущность для верхнеуровневой агрегации счетов)
- Пользовательских отчётов (баланс, движение денежных средств) в виде
 - витрин БД
 - OLAP-куба на базе витрин

3. Характеристики системы

3.1 Словарь

Клиент — вторая сторона сделки (это может быть торговая точка (для торговых операций), банк (например для начисления процентов))

Договор — агрегат операций (например, покупке в магазине соответствует одна сделка — одна проводка, депозиту в банке соответствует несколько проводок: пополнение счёта, начисление процентов, закрытие счёта...)

Проводка — денежная операция, одной из сторон которой является владелец БД. Всегда включает в себя 2 стороны.

Счёт — в общем случае синтетическая сущность, при возможности (например для банковского вклада или уплаты налогов) имеет идентификатор 20-значный принятый в финансовой системе РФ.

3.2 Пользовательские истории

Пользовательская история	Сущности
Я, как владелец, хочу видеть текущее состояние баланса счетов/карт, чтобы контролировать текущее состояние счетов.	Счёт, баланс, отчётная дата
Я, как владелец, хочу иметь видеть приходно/расходные операции по счетам/картам, чтобы оценить их правомерность.	Счёт (счёт карты), операция (проводка)
Я, как пользователь, хочу иметь возможность загружать проводки пулом, чтобы снизить затраты времени на учётные операции.	Проводка, набор проводок, операция загрузки
Я, как владелец, хочу иметь возможность табличной выгрузки данных, чтобы контролировать каждую операцию.	Отчёт, операция (проводка)
Я, как владелец, хочу иметь возможность настройки периода просматриваемых отчётов, чтобы проводить анализ расходов.	Календарный период, баланс

3.3 Описание предметной области

База данных обеспечивает следующие операции:

- Загрузку данных:
 - О состоянии счёта (загрузка, обновление/уточнение)
 - Операциях по счёту
- Хранение данных
 - Данных о счёте (владелец, тип счёта)
 - Операций по счёту (в значении бухгалтерских проводок)
 - Состояния (баланса) счёта на дату
- Формирование отчётов:
 - О состоянии счёта на конкретную дату
 - о прибылях и убытках (баланс приходно/расходных операций на этапе MVP) за заданный период
- Представление отчётов в аналитической системе (OLAP-куб)

База данных обеспечивает следующие технологические операции:

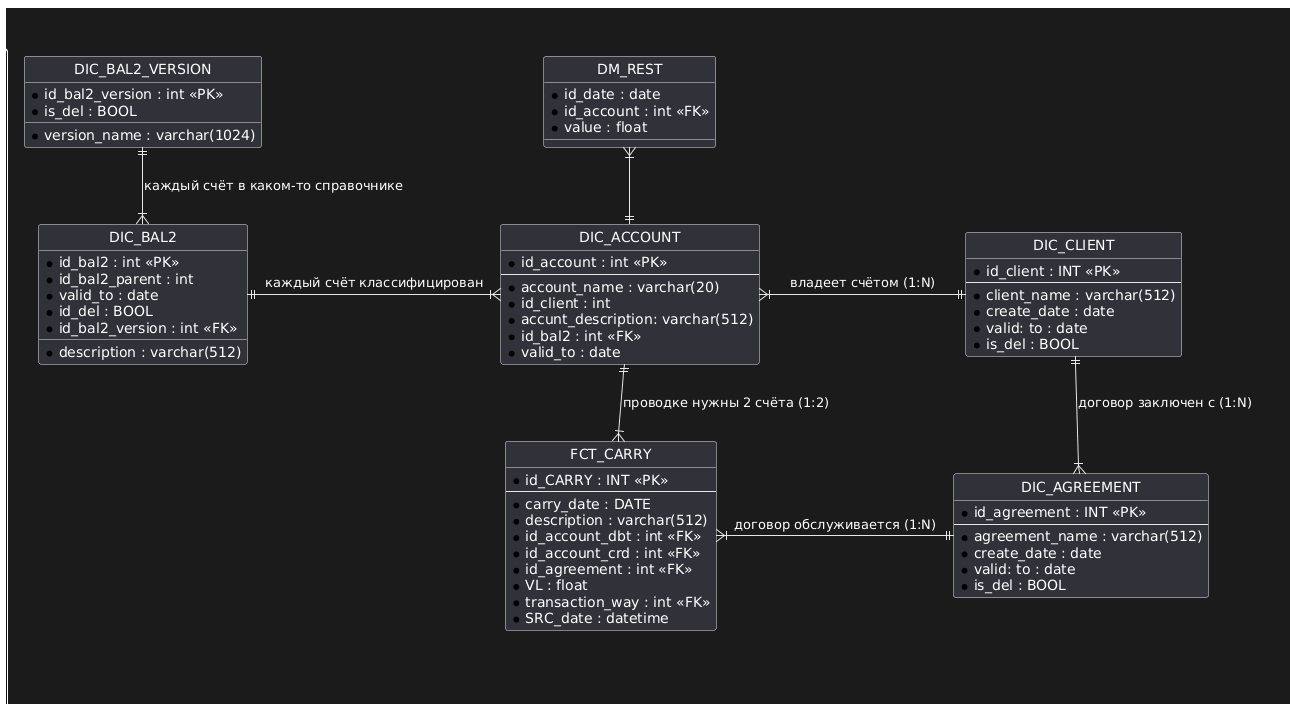
- Валидацию входящих данных в соответствии с заданными параметрами (на этапе MVP: консистентность данных, соответствие типов)
- Расчёт баланса на дату
- Формирование и хранение отчётов

Для пользователя БД должны быть доступны операции:

- Загрузка данных из XLS-файла за период
- Просмотр отчётов (BS, PL)
- Выгрузка ОПУ за период
- Выгрузка баланса на дату
- Выгрузка справочников (клиенты, договоры, счета)

3.4 ER-диаграмма

На этапе MVP ER-диаграмму можно представить в следующем виде:



Текст диаграммы в формате plantUML:

@startuml

'Диаграмма ERD MVP-проекта

'v.1.2

'left to right direction

'Скрытие теней и кружков

hide circle

hide spot

'Определение сущностей

'Счёт, всегда на 1 клиента, второй - владелец базы

'Номер счёта 20-значник, может быть синтетический

Entity DIC_ACCOUNT{

* id_account : int <<PK>>

--

* account_name : varchar(20)

* id_client : int

* account_description: varchar(512)

* id_bal2 : int <<FK>>

* valid_to : date

}

'Клиент (справочник)

Entity DIC_CLIENT

{

* id_client : INT <<PK>>

--

* client_name : varchar(512)

* create_date : date

* valid: to : date

* is_del : BOOL

}

'Договор (справочник)

Entity DIC_AGREEMENT

{

* id_agreement : INT <<PK>>

--

```

* agreement_name : varchar(512)
* create_date : date
* valid_to : date
* is_del : BOOL
}

```

'Проводка (основной поток данных, все поля обязательные)

Entity FCT_CARRY

```

{
  * id_CARRY : INT <<PK>>
  --
  * carry_date : DATE
'описание проводки
  * description : varchar(512)
  * id_account_dbt : int <<FK>>
  * id_account_crd : int <<FK>>
  * id_agreement : int <<FK>>
  * VL : float
  * transaction_way : int <<FK>>
'дата создания проводки
  * SRC_date : datetime
'пользователь создавший проводку
  * SRC_user : int <<FK>>
'основание для создания проводки (из справочника)'
  * carry_ground : int <<FK>>
}

```

'Остаток на дату (таблица заполняемая расчётными процедурами)

Entity DM_REST

```

{
  * id_date : date
  * id_account : int <<FK>>
  * value : float
}

```

'Классификаторы

'Справочник балансовых счетов, версионный

Entity DIC_BAL2

```

{
  * id_bal2 : int <<PK>>
  * description : varchar(512)
  * id_bal2_parent : int
  * valid_to : date
  * id_del : BOOL
  * id_bal2_version : int <<FK>>
}

```

'версии справочника (например разные планы счетов)

Entity DIC_BAL2_VERSION

```

{
  * id_bal2_version : int <<PK>>
  * version_name : varchar(1024)
  * is_del : BOOL
}

```

'Статьи баланса (для отчётов)

'Статьи ОПУ (для отчётов)

'Балансовые счета (для классификаторов)

'-----Связи

'Клиент - договор, счёт

DIC_CLIENT ||-left-|{ DIC_ACCOUNT : владеет счётом (1:N)

'второй стороной договора всегда рассматривается владелец базы для которого она ведётся
DIC_CLIENT ||--|{ DIC_AGREEMENT : договор заключен с (1:N)

'Проводка - счёт (движение между счетами)

DIC_ACCOUNT ||-down-|{ FCT_CARRY : проводке нужны 2 счёта (1:2)

DIC_BAL2 ||-|{ DIC_ACCOUNT : каждый счёт классифицирован

DIC_BAL2_VERSION ||-down-|{ DIC_BAL2 : каждый счёт в каком-то справочнике

DIC_AGREEMENT ||-left-|{ FCT_CARRY : договор обслуживается (1:N)

'Остаток на дату - счёт (историческая таблица)

DIC_ACCOUNT ||-up-|{ DM_REST

@enduml